

Makara Ramoabi

20240045

C++

1. Scenario Analysis

In a graphical user interface (UI) system, different elements such as buttons, text boxes, and images must be displayed on the screen. Although these elements look different and behave differently, they share common features like position, size, and the ability to draw themselves. A base class called `UIElement` defines a virtual function called `draw()`. Derived classes such as `Button`, `TextBox`, and `Image` override the `draw()` function to provide their specific behavior. When the application runs, all UI elements are stored in a single list of `UIElement` pointers. The program loops through the list and calls `draw()` on each object. Due to polymorphism, the correct function for each object is executed at runtime. This allows the system to be flexible and easily extended. New UI elements can be added without changing existing code, making the application more maintainable and scalable.

2. Concept Research

A pure virtual function is a virtual function in C++ that has no implementation in the base class and is declared using `= 0`. It forces derived classes to provide their own implementation. A class containing at least one pure virtual function is called an abstract class and cannot be instantiated directly. Pure virtual functions are essential for achieving runtime polymorphism because they define a common interface for related classes. They ensure consistency across derived classes while allowing each class to implement behavior differently. This promotes code flexibility, extensibility, and proper object-oriented design in large software systems.

3. Tool Practice

```
#include <iostream>
```

```
using namespace std;
```

```
class Shape {
```

```
public:
```

```
    virtual void draw() = 0; // Pure virtual function
```

```
};
```

```
class Circle : public Shape {
```

```
public:
```

```
    void draw() override {
```

```
        cout << "Drawing a Circle" << endl;
```

```
    }
```

```
};
```

```
class Rectangle : public Shape {
```

```
public:
```

```
    void draw() override {
```

```
        cout << "Drawing a Rectangle" << endl;
```

```
    }
```

```
};
```

```
int main() {
```

```
    Shape* shapes[2];
```

```
    Circle c;
```

```
    Rectangle r;
```

```
    shapes[0] = &c;
```

```
    shapes[1] = &r;
```

```
    for (int i = 0; i < 2; i++) {
```

```
        shapes[i]->draw(); // Polymorphic call
```

```
    }
```

```
    return 0;
}
```

Reflection

Writing this polymorphic program in Visual Studio Code helped me understand how runtime polymorphism works in C++. Using a base class pointer to call derived class functions demonstrated dynamic binding clearly. The pure virtual function ensured that all derived classes implemented the `draw()` method. Compiling and running the program showed how the correct function was chosen at runtime. This exercise improved my understanding of abstract classes, inheritance, and virtual functions. It also showed how polymorphism allows flexible and scalable program design, especially when working with collections of related objects in real-world applications.

4. UML Design

The UML diagram begins with an abstract class named `Shape` at the top. The `Shape` class contains a pure virtual method `draw()`. Below it are derived classes such as `Circle` and `Rectangle`, connected with inheritance arrows pointing toward `Shape`. Each derived class implements the `draw()` method. The diagram shows that `Shape` cannot be instantiated, while `Circle` and `Rectangle` can. This hierarchical structure demonstrates polymorphism because objects of different types share a common interface. The UML clearly illustrates the “is-a” relationship, showing that both `Circle` and `Rectangle` are types of `Shape`, enabling runtime polymorphism.